

Slides: PAA_03

Disciplina: Projeto e Análise de Algoritmos.

Professor: André Tavares da Silva.

Aluno: Dalton Solano dos Reis.

Atividade 2 - Slide 29

Elabore um algoritmo recursivo para calcular a raiz quadrada de um número real (positivo) qualquer. Após implementar o algoritmo, escreva a relação de recorrência e a respectiva complexidade do algoritmo para o pior caso.

- Lembrete: comparação de números reais através de aproximação => |x - 3.2| <= 0.00001
- OBS: $r = \sqrt{n}$
 - quando $n \geq 1, 1 \leq r \leq n$
 - quando $n < 1, n \leq r \leq 1$

Resposta:

- no final.

```
In [1]: def raiz_quadrada_recursiva(n, eps=1e-5):
        """Calcula sqrt(n) recursivamente por busca binária."""
        if n < 0:
            raise ValueError("0 algoritmo é definido para números reais positivos.")
        if n == 0:
            return 0.0

        # Intervalo inicial conforme o enunciado.
        if n >= 1:
            esquerda, direita = 1.0, float(n)
        else:
            esquerda, direita = float(n), 1.0

        def busca(esq, dir_):
            meio = (esq + dir_) / 2.0
            aproximacao = meio * meio

            if abs(aproximacao - n) <= eps:
                return meio

            if aproximacao > n:
                return busca(esq, meio)
            return busca(meio, dir_)

        return busca(esquerda, direita)

# Exemplo de uso:
valor = 10.0
resultado = raiz_quadrada_recursiva(valor)
print(f"sqrt({valor}) ~= {resultado}")

sqrt(10.0) ~= 3.1622767448425293
```

Relação de recorrência e complexidade (pior caso)

Como o algoritmo usa busca binária no intervalo $[a, b]$, a cada chamada recursiva o tamanho do intervalo é reduzido pela metade.

Seja $L = b - a$ o tamanho do intervalo atual. No pior caso:

$$T(L) = T\left(\frac{L}{2}\right) + \Theta(1), \quad T(\varepsilon) = \Theta(1)$$

onde $\varepsilon = 0.00001$ é a tolerância de parada.

Logo, o número de chamadas recursivas no pior caso é:

$$k = \left\lceil \log_2\left(\frac{L_0}{\varepsilon}\right) \right\rceil$$

com L_0 sendo o tamanho do intervalo inicial ($n - 1$, quando $n \geq 1$, ou $1 - n$, quando $0 < n < 1$).

Portanto, a complexidade de tempo no pior caso é:

$$T(n) = \Theta\left(\log\left(\frac{L_0}{\varepsilon}\right)\right)$$

Para ε fixo (como no enunciado), isso se comporta como custo logarítmico no tamanho do intervalo inicial.